

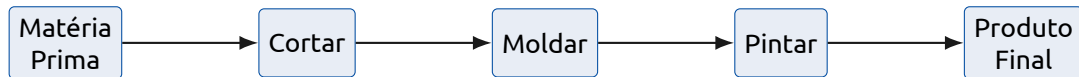
Padrão Algorítmico: Fluxo de Dados

COMP0496 – Programação A

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`



Dados fluem da mesma forma: entram brutos, passam por etapas de transformação e saem prontos para uso. Cada etapa faz **uma coisa** e entrega para a próxima.

Fluxo de Dados

Dados passam sequencialmente por funções independentes:

Fonte → [Filtro] → [Transformação] → [Agregação] → **Saída**

Cada etapa recebe dados, processa e entrega o resultado para a próxima. As funções não dependem de estado externo — apenas da entrada que recebem.

Vantagem: cada etapa pode ser testada e entendida de forma isolada.

Pipeline: notas brutas → filtrar aprovados → calcular média → exibir

```
1  def filtrar_aprovados(notas):
2      return [n for n in notas if n >= 5.0]
3
4  def calcular_media(notas):
5      return sum(notas) / len(notas) if notas else 0.0
6
7  def exibir_resultado(media):
8      print(f"Media dos aprovados: {media:.2f}")
9
10 # Pipeline
11 notas = [3.5, 7.0, 4.8, 9.2, 5.0, 2.1]
12 aprovados = filtrar_aprovados(notas)
13 media = calcular_media(aprovados)
14 exibir_resultado(media)    # Media dos aprovados: 7.07
```

O mesmo resultado pode ser escrito de duas formas:

Aninhado (mais compacto, menos legível):

```
1  exibir_resultado(calcular_media(filtrar_aprovados(notas)))
```

Com variáveis intermediárias (mais legível):

```
1  aprovados = filtrar_aprovados(notas)
2  media      = calcular_media(aprovados)
3  exibir_resultado(media)
```

Prefira variáveis intermediárias com nomes descritivos — o código documenta o fluxo.

Exemplo 2: pipeline de texto



Pipeline: texto bruto → dividir → limpar → normalizar → contar

```
1 import string
2
3 def dividir(texto):
4     return texto.split()
5
6 def limpar(palavras):
7     return [p.strip(string.punctuation) for p in palavras]
8
9 def normalizar(palavras):
10    return [p.lower() for p in palavras]
11
12 def contar(palavras):
13    freq = {}
14    for p in palavras:
15        freq[p] = freq.get(p, 0) + 1
16    return freq
17
18 texto = "Gato, gato e GATO: tres gatos!"
19 freq = contar(normalizar(limpar(dividir(texto))))
20 print(freq)  # {'gato': 3, 'e': 1, 'tres': 1, 'gatos': 1}
```

Fazer:

- Uma responsabilidade por função
- Nomear cada etapa claramente
- Evitar efeitos colaterais nas transformações
- Testar cada etapa de forma isolada

Evitar:

- Funções que fazem filtro e transformação e exibição
- Modificar a entrada dentro da função
- Depender de variáveis globais

Teste isolado: se `filtrar_aprovados([5.0, 3.0, 7.0])` retorna `[5.0, 7.0]`, a etapa está correta — independentemente do restante do pipeline.

- **Fluxo de dados:** dados passam por etapas independentes — Fonte → Transformação → Saída.
- Cada função faz **uma coisa** e recebe/entrega dados sem depender de estado externo.
- Variáveis intermediárias com nomes descritivos tornam o pipeline legível.
- Funções pequenas e focadas são fáceis de **testar isoladamente**.